Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410045	26102000167	BWU/MCA/26/050	Bikram Maity

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



NOVEMBER 2026

Campus Event Management & Feedback Analyzer

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410045	26102000167	BWU/MCA/26/050	Bikram Maity

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

ACKNOWLEDGEMENT

With profound respect and deep gratitude, we express our sincere appreciation to our project supervisor, Mr. Rahul Kumar Ghosh, Assistant Professor, Computational Sciences, Brainware University, for his exceptional guidance, constant supervision, and invaluable suggestions throughout the course of this project. His expertise, constructive feedback, and patience have been instrumental in shaping the direction and successful completion of this work.

I extend my heartfelt thanks to Dr. Jayanta Aich, Head of the Department, Computational Sciences, Brainware University, for providing us with the necessary academic support, facilities, and encouraging environment that enabled us to carry out this project efficiently.

Roll No	Registration No	Student Code	Student Name	Signature
26102410045	26102000167	BWU/MCA/26/050	Bikram Maity	

ABSTRACT

This project presents a Python-based application for Campus Event Management and Feedback Analysis. The system allows administrators to create and manage campus events, register students, track participation, collect feedback ratings and comments, and generate statistical insights and visual charts. It is developed using modular Python scripts (main.py, functions.py, analysis.py, models.py) and relies on libraries such as pandas, matplotlib, and numpy for data handling and visualization.

A simple command-line interface enables users to create events, add students, register participants, cancel registrations, submit feedback, view statistics, and generate graphical reports without complex coding. The system computes average ratings, rating distributions, event-wise participation, and produces bar charts and histograms. This reduces manual effort for campus event organizers and demonstrates practical use of Python data analysis and visualization techniques in an educational management context.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	5
5	System Flow	6
6	Proposed Design	13
7	Experimental Result	15
8	Future Scope	18
9	Conclusion	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	7
5.3	Data Flow Diagram Level 1	8
5.4	Use Case Diagram	9
5.5	Activity Diagram	10
5.6	Sequence Diagram	11
5.7	Flow Chart	12
7.1	Main Menu Interface	15
7.2	Event Creation & Listing	15
7.3	Feedback Analysis	16
7.4	Bar Chart Generation	16
7.5	Histogram Generation	17
7.6	Participation Chart	17

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

Symbol / Abbreviation	Description
CSV	Comma Separated Values (file format used for events, students, registration data input
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
CLI	Command Line Interface
ID	Identifier (Event ID / Student ID / Registration ID)
Avg	Average (used for Average Rating)
%	Percentage (used where applicable in participation)
→	Data flow / Message direction (in DFD and Sequence Diagram)
○ / ▭	Process / Activity symbol (in Flow Chart and Activity Diagram)
◇	Decision symbol (in Flow Chart and Activity Diagram)
▭ (open)	Data Store (in DFD)
Actor	External entity / User (in Use Case and Context diagrams)

CHAPTER 1: INTRODUCTION

Traditional campus event management relies on manual registers, spreadsheets, paper forms, and fragmented communication channels. Organizers often struggle to track registrations, enforce capacity limits, collect structured feedback, and generate summary reports or visual insights. Calculating participation statistics and rating distributions requires additional manual effort.

With the availability of Python data analysis libraries, it is now possible to automate large parts of this process. This project focuses on building a system that manages campus events, student registrations, and feedback through a modular command-line application. The system reads and writes structured CSV files, creates Event and Student objects, handles registration logic with capacity checks, collects ratings (1–5) with comments, computes statistical metrics, and produces clear visualizations.

The system uses a modular design with separate files for data modeling (models.py), core operations (functions.py), analysis and chart generation (analysis.py), and the main program entry point (main.py). Behind the interface, the workflow follows clear steps: load data, validate records, perform CRUD-style operations on events and registrations, collect feedback, generate statistics, produce charts, and present consolidated insights. The project demonstrates how data processing, object-oriented modeling, statistical calculation, and visualization can be combined into a lightweight, easy-to-understand campus management tool suitable for students and beginners.

CHAPTER 2: OBJECTIVE

The objective of this project is to develop a Python-based Campus Event Management & Feedback Analyzer that enables users to manage campus events, register students, collect feedback, and obtain meaningful statistical and visual insights. The system aims to provide a modular, automated workflow by integrating data modeling, operational functions, metric calculation, and visualization components.

Users can create and view events, add students, register or cancel registrations (with capacity enforcement), submit feedback ratings and comments, compute average ratings and rating distributions, analyze event-wise participation, and generate bar charts and histograms without requiring advanced programming knowledge. By combining custom classes for Event, Student and Registration, pandas for data handling, and matplotlib-based visualizations, the project intends to simplify campus event monitoring, reduce manual calculation effort, and offer an accessible end-to-end event and feedback analysis pipeline.

CHAPTER 3: PLANNING

The planning phase outlines the step-by-step approach used to design, organize, and implement the Campus Event Management & Feedback Analyzer. Proper planning ensures that the workflow remains efficient, modular, and easy for beginners to understand and use.

1. Requirement Identification

- Understand the need for a simple Python tool to manage campus events, student registrations, and feedback from CSV data.
- Identify the target users (event organizers, teachers, student coordinators, academic administrators, beginners).
- Define the core requirement: Event & Student data → Registration management → Feedback collection → Statistics + Charts.

2. System Design Planning

- Decide to use object-oriented classes for Event, Student, and Registration.
- Use pandas for data loading/analysis and matplotlib for visualization.
- Choose a modular structure with separate files for models, operations, analysis/charts, and main program.
- Plan the workflow from data loading to final reports and charts.

3. Pipeline Structure

Design a sequential pipeline with the following steps:

- Data loading from CSV files (events, students, registrations, feedback)
- Object creation / data validation
- Event creation and viewing
- Student addition and registration (with capacity checks)
- Feedback submission (rating 1–5 + comment)
- Statistical summaries (average rating, distribution, participation)
- Chart generation (bar chart, histogram, participation chart)
- Result display

4. File & Module Planning

- main.py – Entry point and menu-driven user interaction
- data/ – CSV files for persistent storage
- charts/ – Generated chart images

5. Code Structure Planning

- Break the code into clear, reusable sections: data loading/validation, CRUD operations, metric calculation, filtering, chart plotting, and main menu.

6. Testing & Validation Plan

- Test with sample CSV data.
- Verify capacity enforcement, duplicate registration prevention, and rating range validation.

- Check chart generation for rating distribution and participation.
- Validate handling of edge cases (empty data, full events, missing files).

7. Scalability & Improvement Planning

- Support for multiple campuses or departments
- Export of reports to PDF or Excel
- Interactive GUI (Tkinter or Streamlit)
- Sentiment analysis on feedback comments
- Database integration instead of CSV

CHAPTER 4: REQUIREMENT ANALYSIS

Functional Requirements

1. Data Input

- The system must read and write event, student, registration, and feedback records from CSV files under the data/ directory.
- Each record includes identifiers, names, dates, venues, capacities, ratings, and comments as applicable.
- Input is handled through Python file operations and pandas DataFrame operations.

2. User Interaction

- A simple command-line interface /menu-driven program in main.py.
- Options to create/view events, add students, register/cancel, give feedback, analyze, show charts, and exit.
- Users should not need to write code to obtain results.

3. Model and Framework Requirements

The project depends on the following Python libraries:

- pandas (data loading and manipulation)
- matplotlib (chart generation)
- numpy (numerical calculations)
- Custom modules: models.py, functions.py, analysis.py

4. Analysis Requirements

- Calculate average rating across all feedback.
- Produce rating distribution (count of 1–5 ratings).
- Compute total feedback count and event-wise average ratings.
- Calculate event-wise participation counts from registrations.
- Identify capacity status and registered students per event.

5. Visualization Requirement

- Generate bar charts for feedback rating distribution.
- Generate histograms for rating frequency.
- Generate participation charts (event-wise).
- Charts should be saved under charts/ and displayed.

6. Output Organization

- Display textual summaries and statistics on the console.
- Generate and save chart images.
- Provide clear final reports of event-level and overall insights.

CHAPTER 5: SYSTEM FLOW

Context Level Diagram (CLD):

It shows the overall system as a single process interacting with external entities (User and CSV data stores) and producing reports/charts.

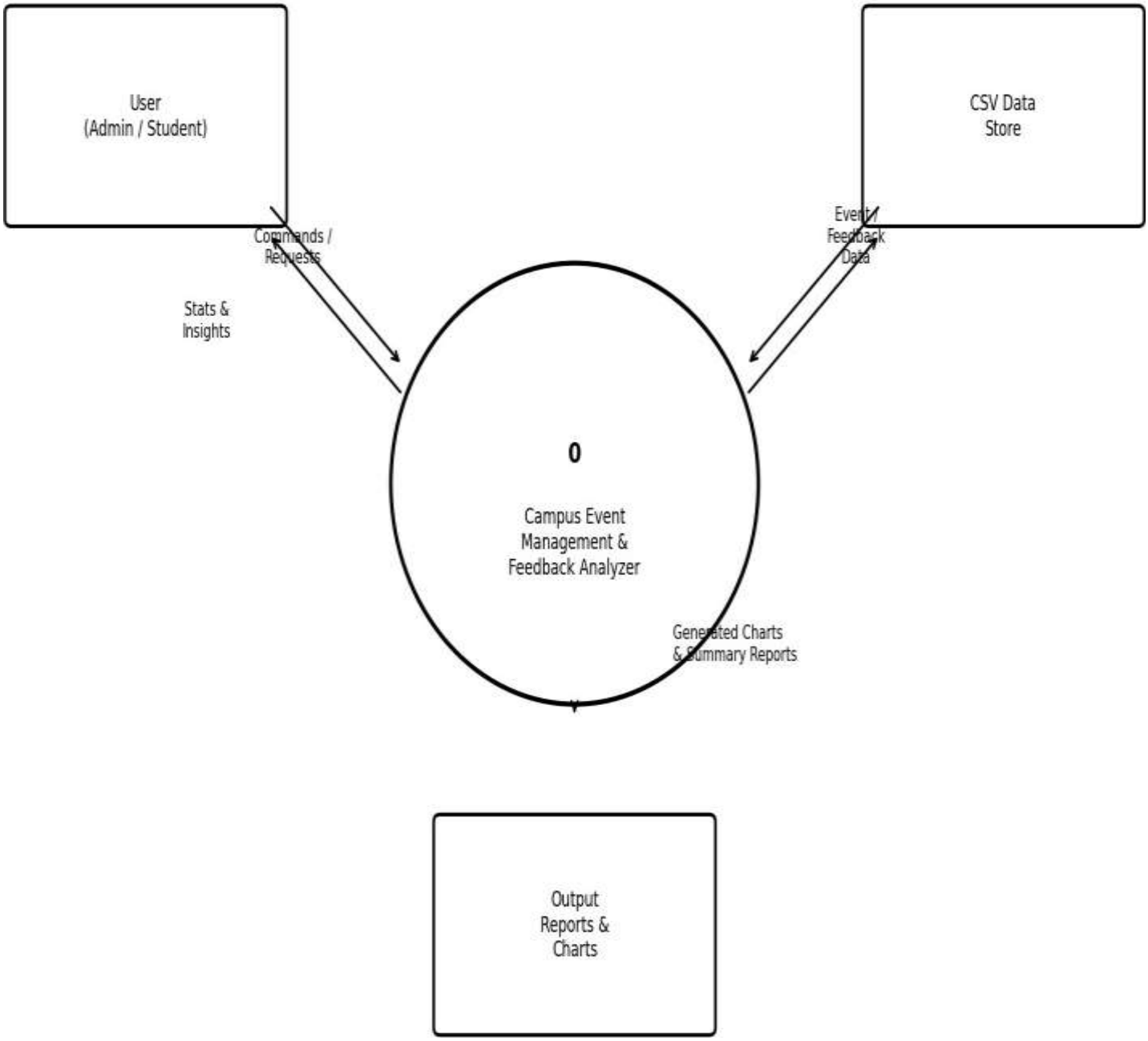


Fig 5.1: Context Level Diagram

Data Flow Diagrams (DFD):

It illustrates the major processes (Manage Events, Manage Students, Handle Registrations, Collect Feedback, Analyze & Visualize) along with data stores and high-level data flow.

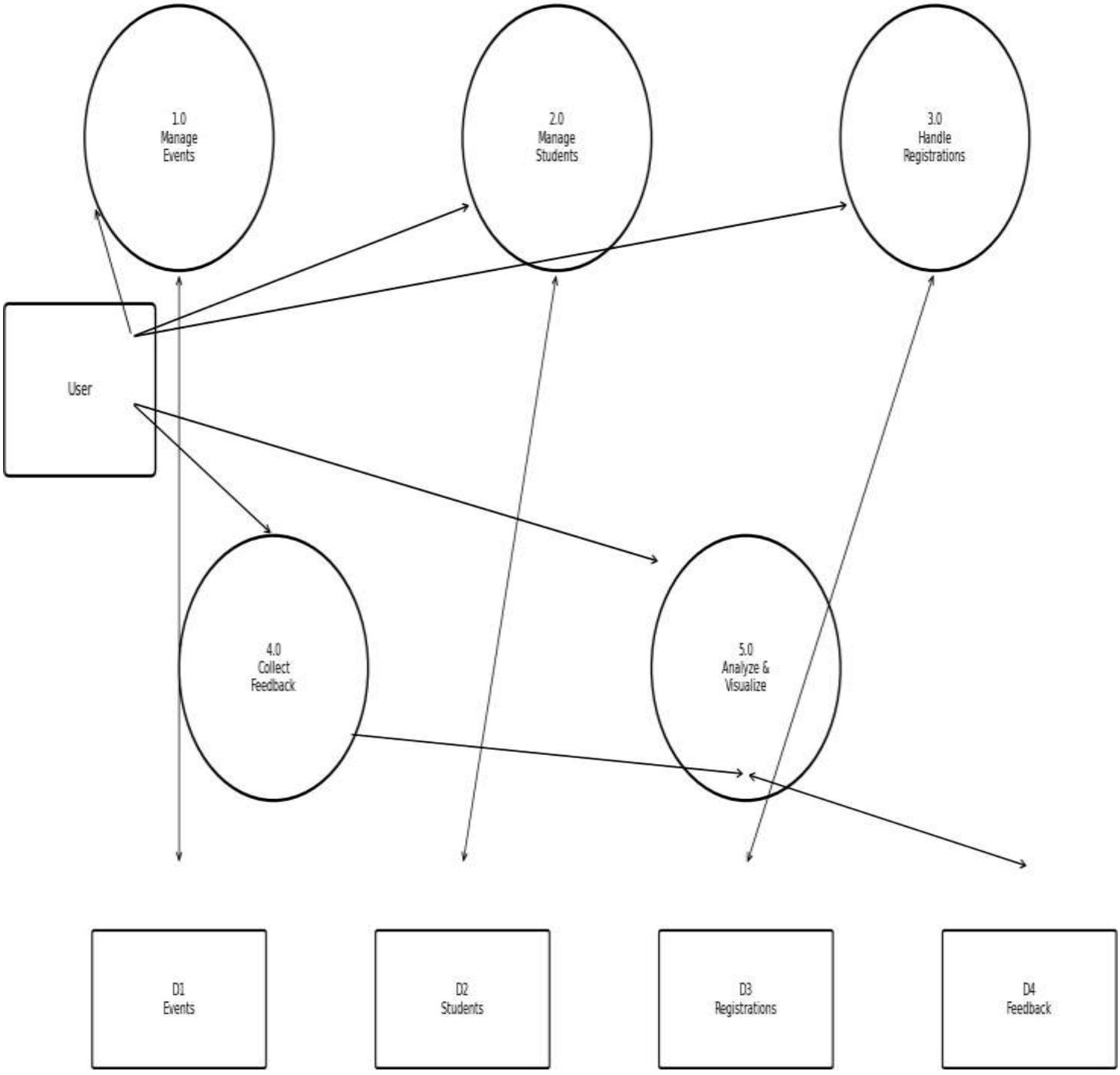


Fig 5.2: Data Flow Diagram Level 0

It breaks down the Level 0 processes into detailed sub-processes for event creation, student management, registration handling, feedback collection, statistics, and chart generation.

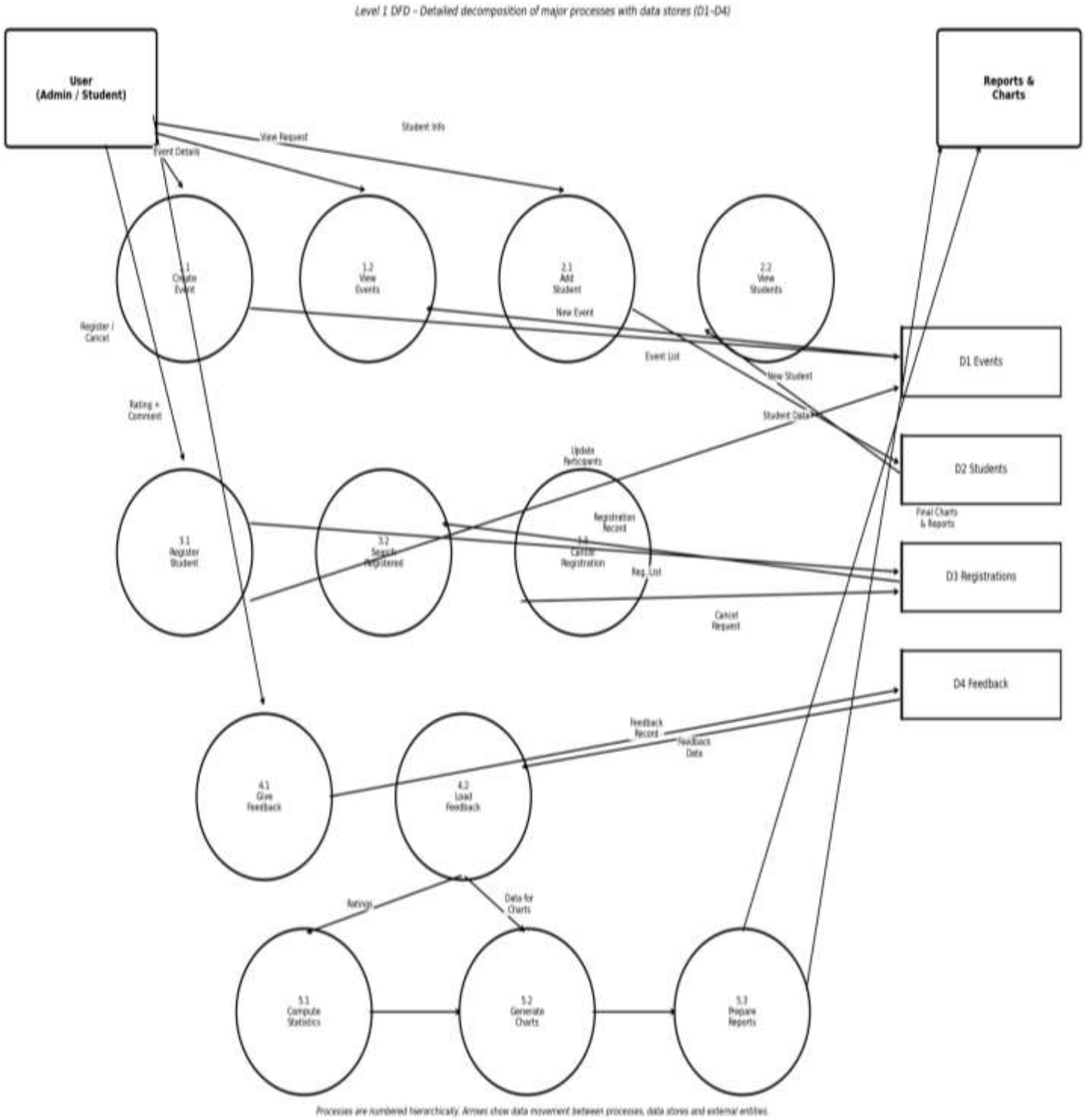


Fig 5.3: Data Flow Diagram Level 1

Use Case Diagram:

It depicts the interactions between the Teacher/Administrator and the system's key functionalities such as creating events, registering students, collecting feedback, and generating analysis reports.

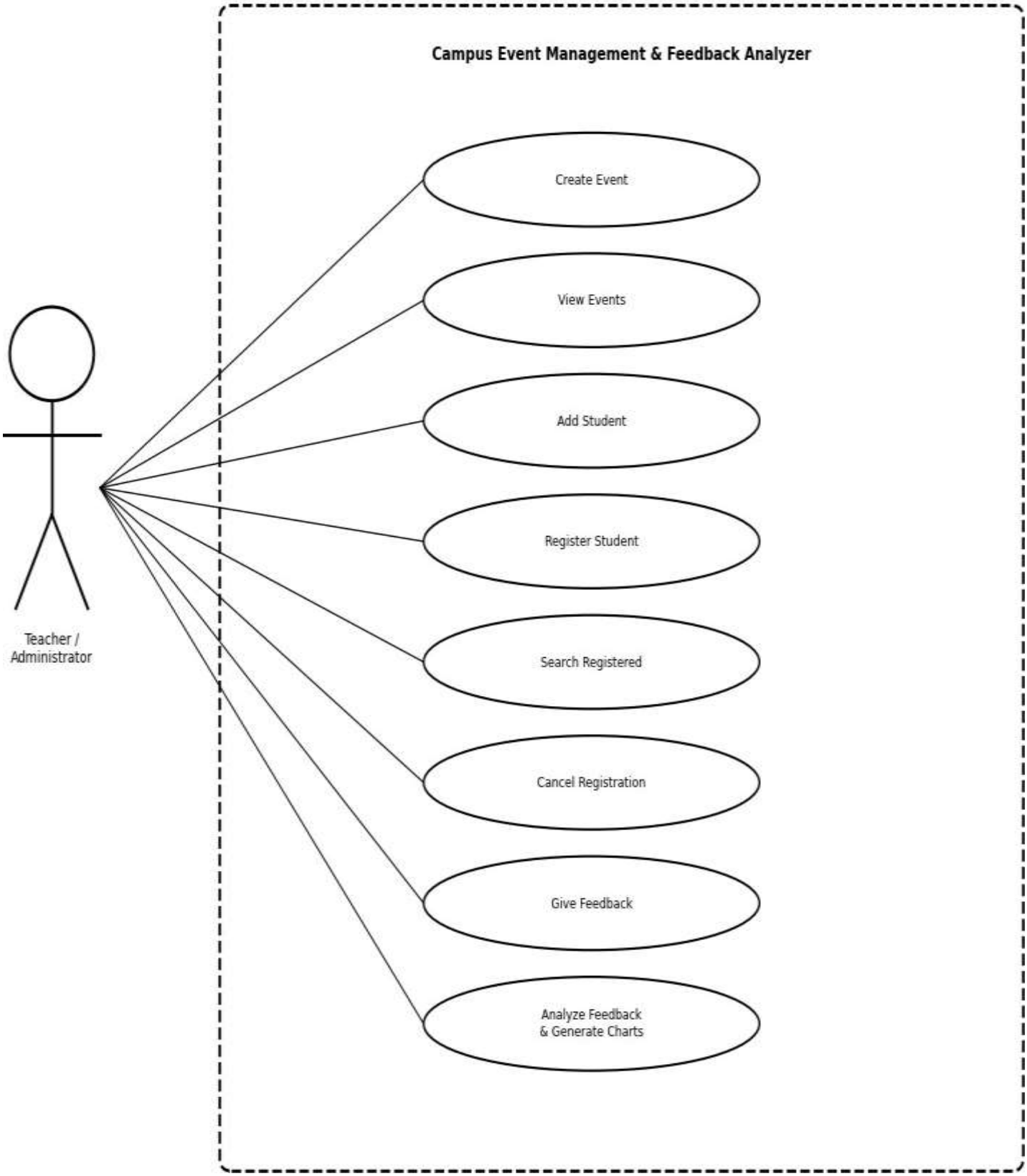


Fig 5.4: Use Case Diagram

Activity Diagram:

It represents the step-by-step workflow of the system from starting the application to displaying the final results, including decision points for menu selection and exit.

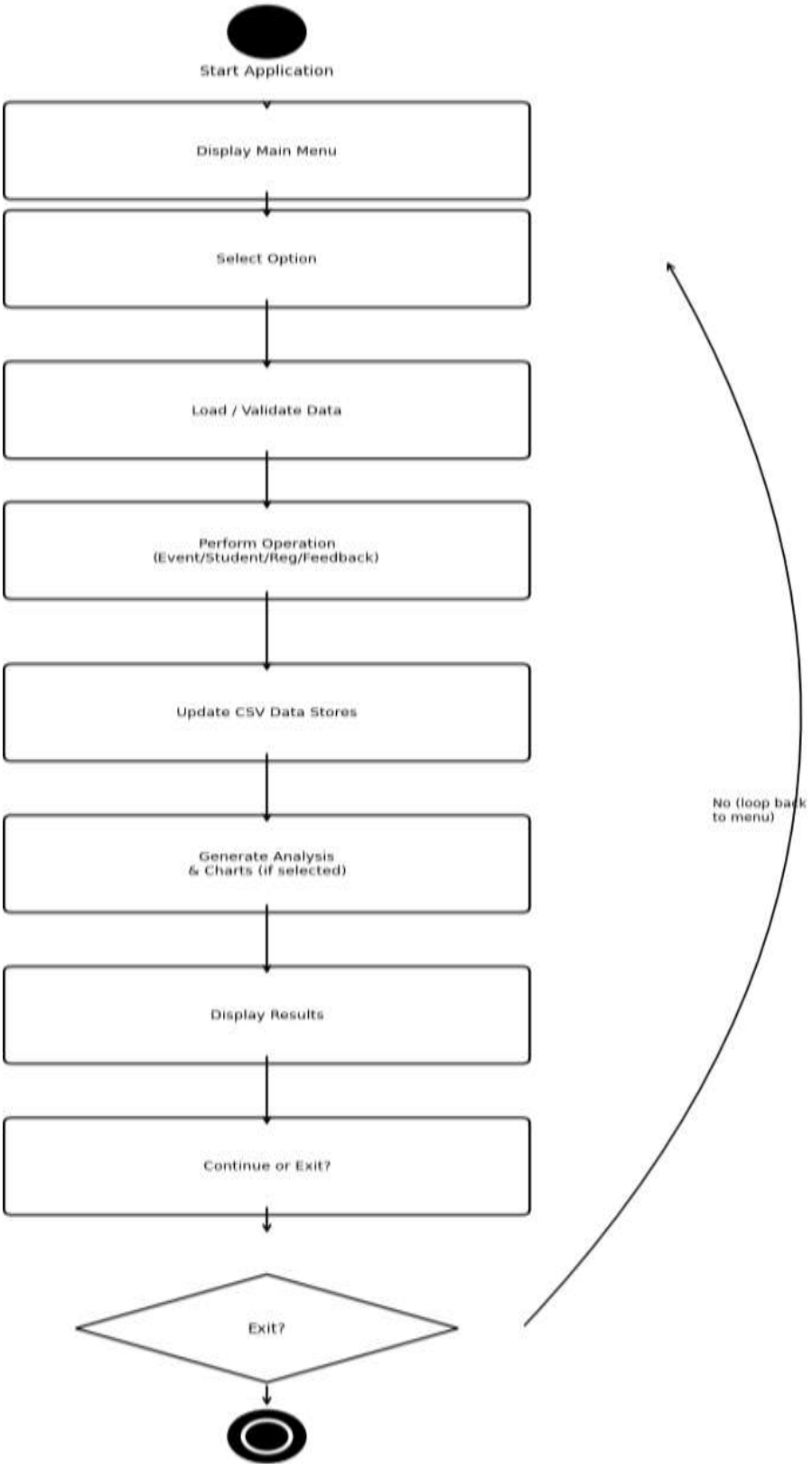


Fig 5.5: Activity Diagram

Sequence Diagram:

It shows the chronological sequence of messages exchanged between the User, Main Controller, Functions Module, Models, Analysis Module, and CSV Data Store.

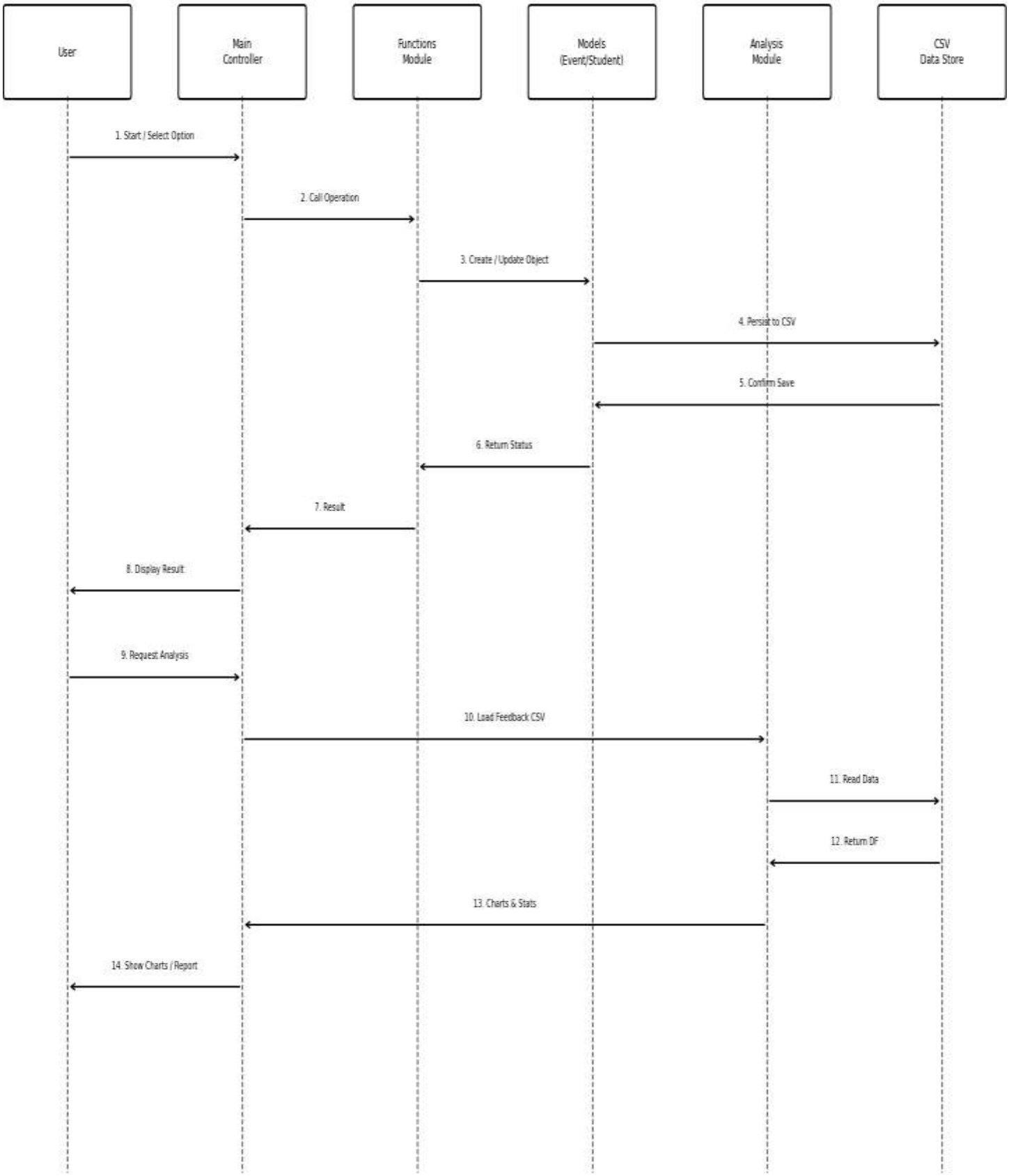


Fig 5.6: Sequence Diagram

Flow Chart:

It presents the complete sequential logic of the system using standard flowchart symbols, including decision points for valid choice, analysis selection, and exit path.

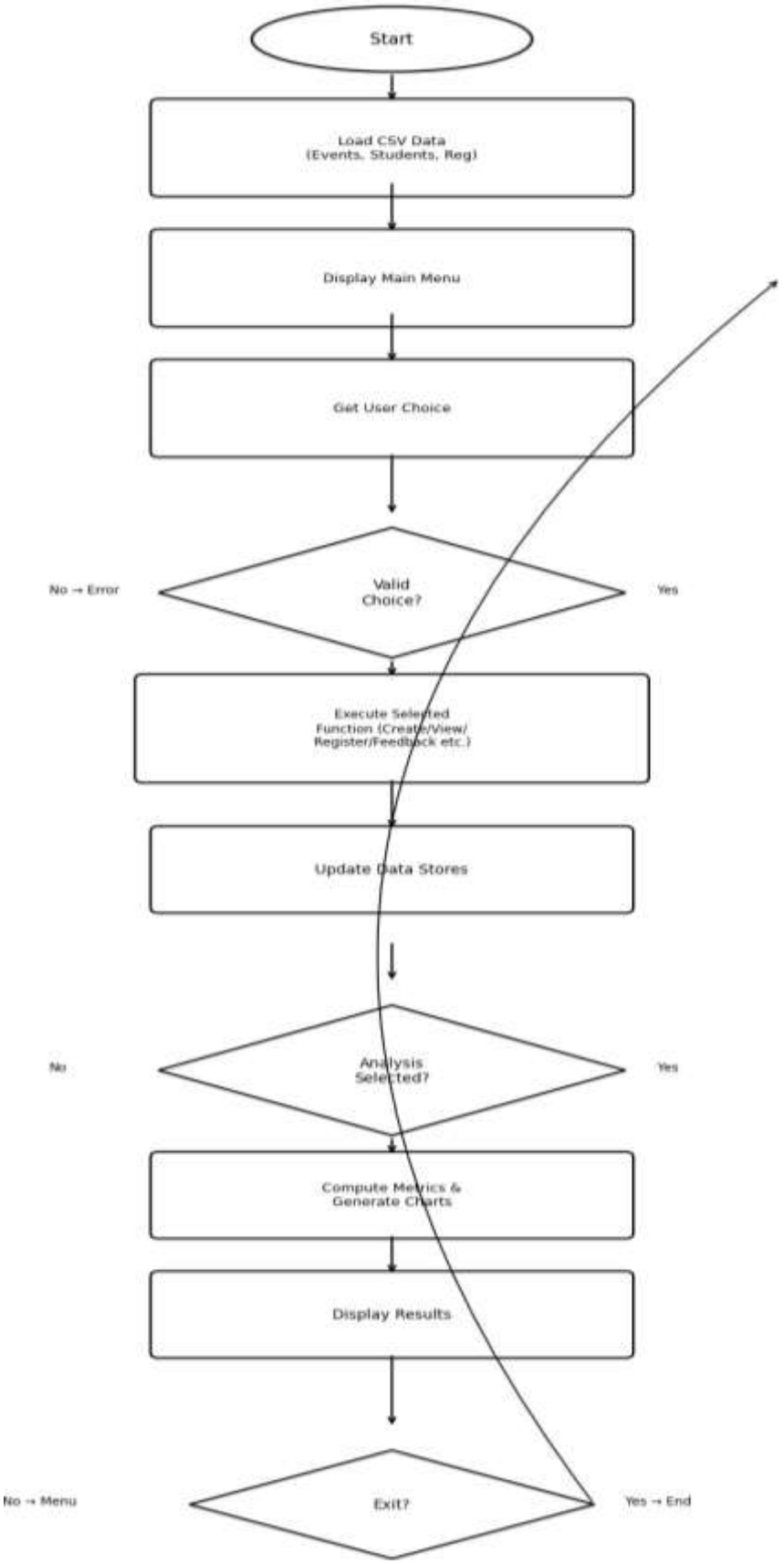


Fig 5.7: Flow Chart

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the system processes event and feedback data and produces analysis results and charts. The method follows a sequential pipeline that begins with CSV input and ends with statistics and visualizations.

The application loads event, student, registration and feedback records from CSV files, creates structured objects, enforces business rules (capacity, uniqueness), collects ratings, computes metrics, and generates charts. This method allows users to obtain insights from campus event data without manual calculations or complex tools.

The process includes the following key stages:

1. Data loading from CSV
2. Object modeling (Event, Student, Registration)
3. Event and registration operations
4. Feedback collection
5. Statistical summary generation
6. Chart creation and display
7. Final report presentation

Step 1: Data Loading

The program reads CSV files (events.csv, students.csv, registrations.csv, feedback.csv) using pandas or standard CSV handling and prepares clean datasets / object lists for operations.

Step 2: Modeling

models.py defines the Event class (with participants list and capacity checks), Student class, and Registration class. Objects are created and maintained in memory and synchronized with CSV stores.

Step 3: Core Operations

functions.py implements create_event, view_events, add_student, register_student (with full-capacity and duplicate checks), search_registered_students, cancel_registration, and give_feedback (rating validation 1–5).

Step 4: Analysis

analysis.py performs average rating calculation, rating distribution, total feedback count, event-wise participation, and feedback summary reports.

Step 5: Chart Generation

The same module uses matplotlib to produce bar charts of rating distribution, histograms of ratings, and participation charts. Images are saved under the charts/ directory.

Step 6: Main Program Flow

main.py orchestrates the process via a numbered menu: it calls the appropriate functions from functions.py or analysis.py based on user choice and presents results through the console.

REPRESENTATIVE CODE STRUCTURE (based on project files)

- main.py: Entry point that presents the menu and dispatches calls.
- data/: Sample / runtime CSV datasets.
- charts/: Output directory for generated images.

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how events and registrations were managed, how feedback metrics were calculated, and how charts were generated. Representative console outputs and chart descriptions are summarized below.

1. Main Menu Interface

The system presents a clear numbered menu for all operations: Create Event, View Events, Add Student, Register Student, Search Registered Students, Cancel Registration, Give Feedback, Analyze Feedback, Show Charts, and Exit.

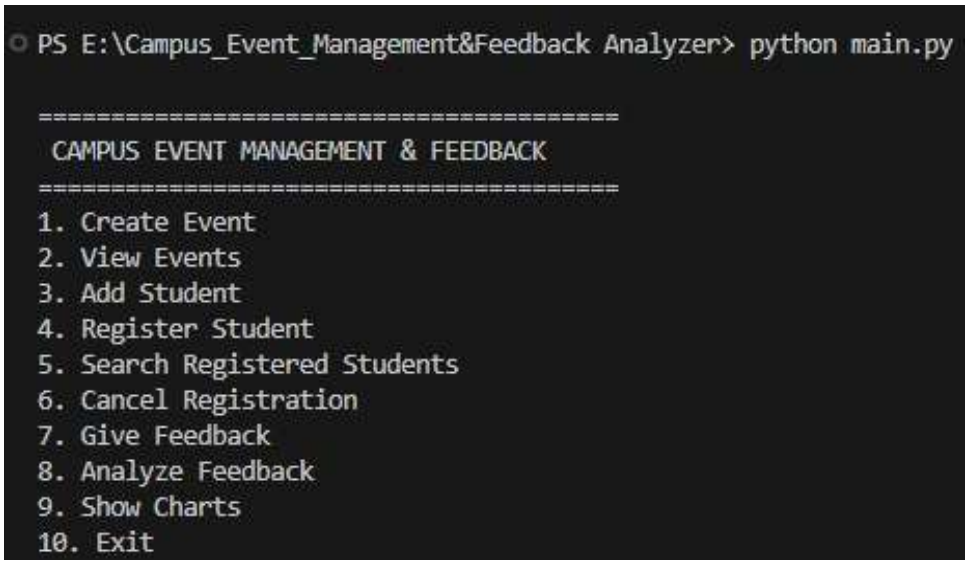


Fig 7.1: Main Menu Interface

2. Event Creation & Listing

Events are successfully created with ID, name, date, venue and maximum participants. View Events displays current registration counts against capacity.

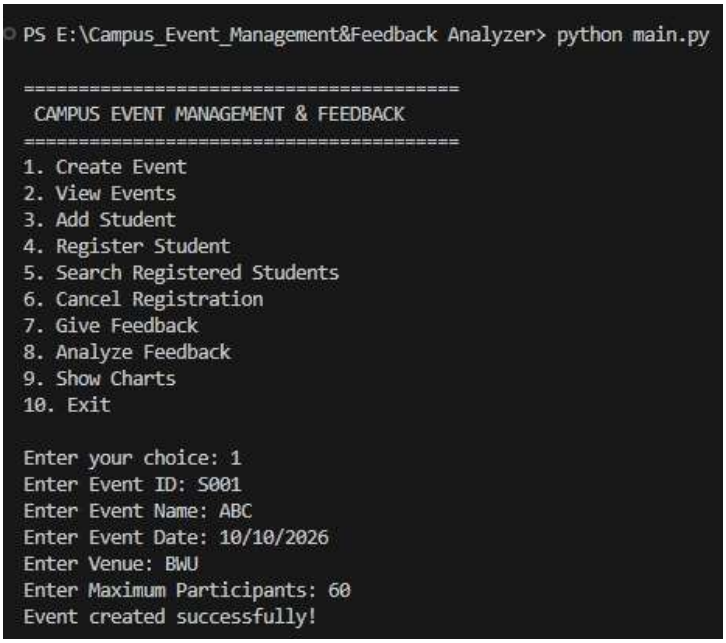


Fig 7.2: Event Creation and Listing

3. Feedback Analysis

Average rating, rating distribution (value counts sorted by rating 1–5), total feedback count, and event-wise average ratings are computed and printed clearly.

```
========================================
 CAMPUS EVENT MANAGEMENT & FEEDBACK
========================================
1. Create Event
2. View Events
3. Add Student
4. Register Student
5. Search Registered Students
6. Cancel Registration
7. Give Feedback
8. Analyze Feedback
9. Show Charts
10. Exit

Enter your choice: 7
Enter Event ID: S001
Enter Student ID: B001
Enter Rating (1-5): 4
Enter Feedback Comment: Good
Feedback submitted successfully!
```

Fig 7.3:Feedback Analysis

4. Charts Generation

Visual charts (Feedback Rating Distribution bar chart, Rating Histogram, and Event Participation chart) are produced and saved under the charts/ folder. These visualizations help organizers quickly understand rating patterns and participation levels.

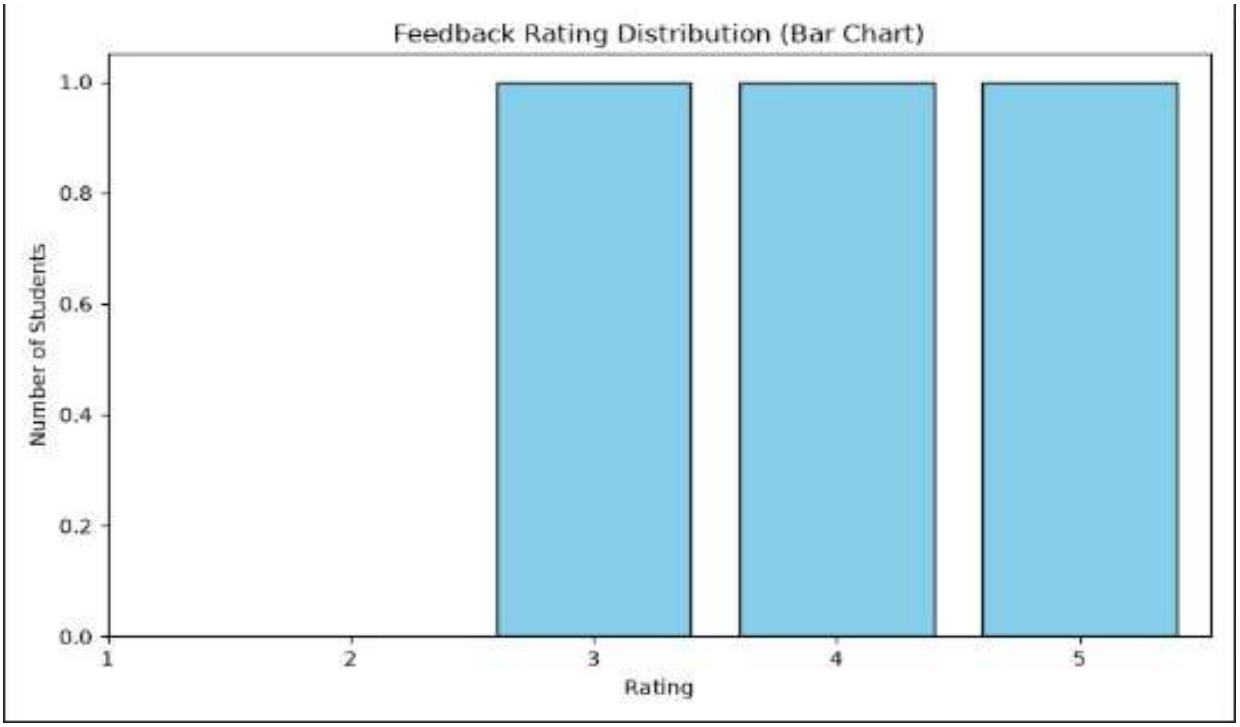


Fig 7.4: Bar Chart Generation

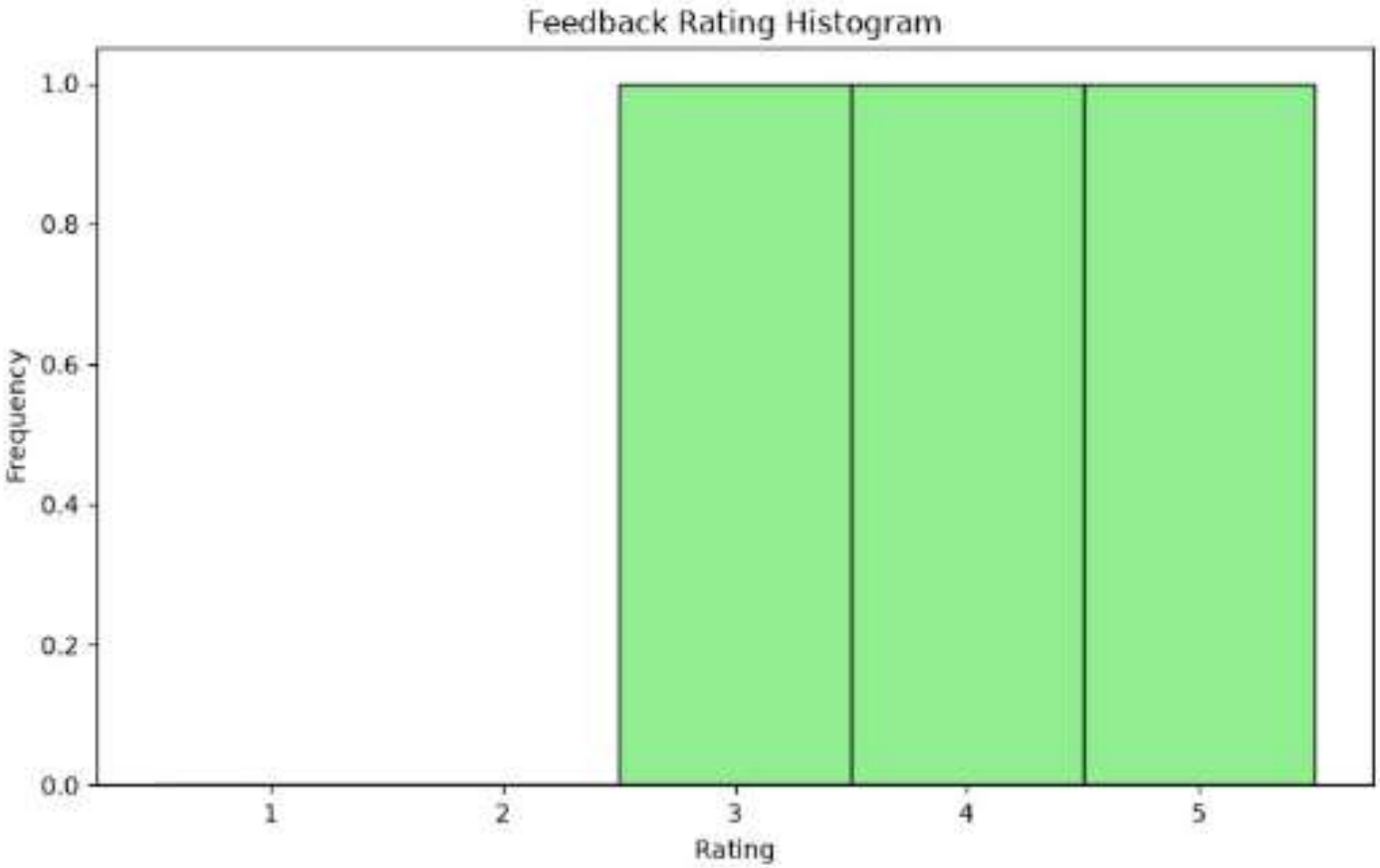


Fig 7.5: Histogram Generation

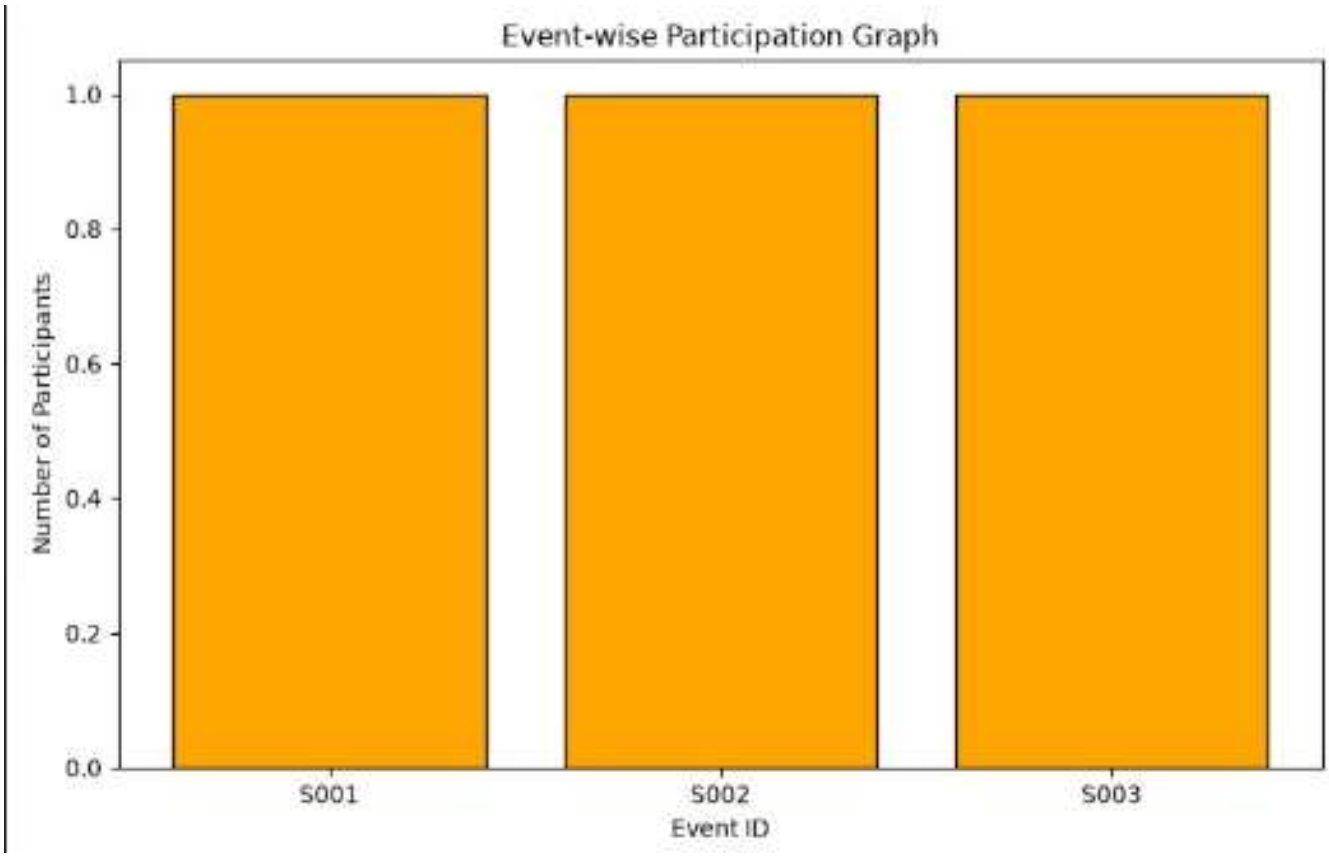


Fig 7.6: Participation Chart

CHAPTER 8: FUTURE SCOPE

Several improvements can be added in later versions of the project:

1. Enhanced User Interface

Develop a graphical user interface (Tkinter, Streamlit, or web-based dashboard) for easier data entry, interactive filtering, and real-time visualization.

2. Multi-Campus / Multi-Department Support

Extend the data model to handle multiple departments, campuses, or event categories with hierarchical reporting.

3. Report Export Features

Add options to export analysis results and charts to PDF, Excel, or PowerPoint for easy sharing with administrators and stakeholders.

4. Sentiment Analysis on Comments

Incorporate basic NLP techniques to analyze free-text feedback comments for polarity and key themes.

5. Database Integration

Replace or supplement the CSV input with a proper database (SQLite/MySQL) for larger datasets and multi-user access.

6. Notification & Alert System

Add automatic alerts or email notifications for low-rated events, capacity approaching limits, or new feedback submissions.

7. Advanced Visualizations

Include more sophisticated charts, heatmaps, trend lines over time, and interactive dashboards.

CHAPTER 9: CONCLUSION

The project successfully showcases a complete, end-to-end workflow for managing campus events, student registrations, and feedback analysis through an automated and modular Python pipeline. Beginning with CSV input, the system efficiently loads records, enforces business rules, collects structured feedback, computes meaningful metrics (average ratings, distributions, participation counts), and generates clear visualizations. This demonstrates that practical educational event analytics can be achieved using standard Python libraries when the process is divided into clear, modular stages.

Moreover, the modular design (separate files for data modeling, operations, analysis/charts, and main logic) significantly enhances maintainability and usability, allowing individuals with basic Python knowledge to understand and extend the system. The design emphasizes accessibility and practicality, ensuring that event and feedback monitoring is not limited to complex enterprise tools. Overall, the project meets its primary objective by delivering a functional, data-driven Campus Event Management & Feedback Analyzer that is both practical and educationally valuable.

REFERENCES

1. pandas Development Team. pandas: Powerful data structures for data analysis. Available at: https://pandas.pydata.org/

2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering. Available at: https://matplotlib.org/

3. Python Software Foundation. Python Language Reference. Available at: https://www.python.org/

4. Project source code and files. Available at the project Google Drive folder provided with the submission.